

Documento de Diseño del Juego (Mini GDD)

Proyecto: Asteroids

Autor: Diego Javier Garabán Gil

Titulación: Grado en Ingeniería Informática - Universidad Politécnica de Madrid (UPM)

Fecha: 27 de septiembre de 2026

1. Resumen Ejecutivo

Asteroids 2D es un videojuego arcade bidimensional de vista cenital (*top-down*) desarrollado en el motor Unity. El jugador pilota una nave espacial en un entorno cerrado pero infinito gracias a un sistema de envoltura de pantalla. El objetivo principal es sobrevivir el mayor tiempo posible esquivando y destruyendo meteoritos, acumulando la máxima puntuación y evitando penalizaciones por falta de precisión.

2. Bucle de Juego (Core Loop)

La experiencia de juego se basa en un ciclo continuo de acción y reacción:

1. **Aparición:** El sistema genera meteoritos constantemente en los márgenes exteriores de la cámara.
2. **Evasión y Posicionamiento:** El jugador utiliza el movimiento direccional y el teletransporte en los bordes para nunca salir de la pantalla.
3. **Ataque:** El jugador apunta con el ratón y dispara para eliminar los meteoritos.
4. **Recompensa/Castigo:** Si aciertas te suman puntos y si fallas te restan, así hay que jugar con una estrategia.

3. Mecánicas Principales

3.1. Teletransporte Simétrico de Bordes.

Cuando la nave rebasa estos límites, se aplica una inversión simétrica de su coordenada (multiplicación por -1), transportándola de manera instantánea al extremo opuesto del mapa.

3.2. Spawning Dinámico de Obstáculos

La instanciación de los meteoritos no depende de puntos fijos predefinidos. El generador (*Spawner*) calcula los límites reales de la cámara y selecciona aleatoriamente los márgenes exteriores para instanciar los objetos. Esto asegura que los enemigos siempre nazcan fuera

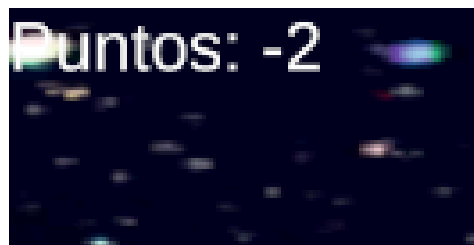
del campo de visión del jugador, evitando apariciones abruptas en pantalla y manteniendo la inmersión.



3.3. Sistema de Puntuación de Precisión

El juego integra un modelo de riesgo-recompensa en el sistema de disparo:

- **Impactos Positivos:** Destruir un meteorito recompensa al jugador incrementando el marcador general.
- **Penalización por Fallo:** Los proyectiles que abandonan los límites de la pantalla sin haber colisionado con un objetivo restan puntos. Esto ayuda a disparar menos indiscriminadamente y exige precisión por parte del usuario.



3.4. Interfaz de Usuario y Control de Flujo

Al accionar el comando de pausa, la escala de tiempo del motor (`Time.timeScale`) se iguala a cero, congelando todas las físicas y actualizaciones. Se despliega una interfaz con dos funciones principales:

- **Reanudar:** Restaura la escala de tiempo a 1, ocultando el menú y devolviendo el control al jugador de forma inmediata.
- **Reiniciar:** Llama al gestor de escenas (`SceneManager`) para recargar el nivel desde cero, restableciendo variables, puntuación y posición de las entidades.



4. Arquitectura de Software y Scripts (C#)

El código se ha estructurado de forma modular:

- **PlayerMovement.cs:**
 - Procesa el vector de entrada (`Input.GetAxis`) para aplicar traslación continua.
 - Resuelve la rotación de la nave hacia el puntero del ratón utilizando funciones trigonométricas (`Mathf.Atan2`) tras convertir las coordenadas de pantalla a coordenadas de mundo (`ScreenToWorldPoint`).
 - Ejecuta la lógica del teletransporte simétrico en cada *frame*.
 - Gestiona la instanciación de los *prefabs* de munición.
- **BulletMovement.cs:**

- Se encarga del ciclo de vida del proyectil. Aplica velocidad constante en línea recta y autodestruye el objeto (*Destroy*) cuando supera los límites de la cámara, aplicando la penalización correspondiente al marcador.
- **EnemySpawner.cs:**
 - Utiliza temporizadores (*Time.deltaTime*) para gestionar el ritmo de aparición.
 - Calcula posiciones aleatorias en los vectores perimetrales para instanciar a los enemigos, garantizando una distribución impredecible pero controlada.

5. Controles y Experiencia de Usuario

La interfaz de control está optimizada para la configuración estándar de PC:

- **Traslación:** Teclas W, A, S, D o flechas direccionales.
- **Rotación y Apuntado:** Seguimiento dinámico del cursor del ratón (ejes X e Y).
- **Acción Principal:** Clic izquierdo del ratón para abrir fuego.



6. Recursos y Assets

- **Gráficos:** *Sprites* 2D importados con configuración de píxeles por unidad (PPU) coherente para mantener la proporción geométrica en todo momento.
- **Físicas:** Uso de componentes *Rigidbody2D* (en modo cinemático) y *BoxCollider2D/CircleCollider2D* con *Triggers* para resolver las intersecciones de impacto sin generar fuerzas de rebote indeseadas.